

AI Short Story Co-Writer with Plot Branching – Detailed Problem Brief

1. Problem Context

Writers often face "blank page paralysis"—staring at a blank document with no idea how to start or where to take a story. Even experienced writers benefit from collaborative brainstorming to explore "what if" scenarios and unexpected plot twists. Current writing tools focus on editing and organization but don't help with *ideation*. Outline generators exist, but they're linear and don't let writers interactively explore branching narratives. There's a gap for an AI co-writer that generates narrative scenes, offers plot choices, and lets writers see all their explored branches visually—turning writing from a solitary struggle into an interactive, explorable creative conversation.

2. Objective

Build a web app where users input a story premise, the AI generates the opening scene, and then presents 2–3 plot directions at each branch point. Users choose a direction, the AI continues the story, and the process repeats. The app visualizes all explored branches in a story tree, and users can export a combined final draft by stitching together their chosen path through the branches.

3. Core Features (Must-Have)

- **Story premise input** – A simple form where users enter a story title, genre, and opening premise (e.g., "A detective discovers a hidden door in her office. Mystery genre. Noir tone."). This sets the context for all AI generations.
- **AI-generated opening scene** – The backend calls an LLM with the premise to generate a compelling 300–500 word opening scene that establishes setting, character, and tension.
- **Branch point with plot choices** – After each scene, the AI generates 2–3 possible plot directions as short summaries (e.g., "Follow the mysterious stranger," "Investigate the door immediately," "Receive a cryptic phone call"). Users click to choose their path.
- **Recursive scene generation** – When a user picks a direction, the AI generates the next scene (300–500 words) that follows from that choice, maintaining narrative coherence. Each scene ends with new plot choices.
- **Story tree visualization** – A visual tree or graph showing all explored branches so far. Nodes represent scenes/decision points, edges represent plot choices. Users can click nodes to view that scene, see which branch is their "current path," and jump between branches to explore alternatives.
- **Story context preservation** – The AI maintains awareness of the full story path (opening + all previous scenes in the chosen branch) so new scenes feel coherent and reference past events. Backend keeps a "current narrative thread" or "conversation history" as users navigate.
- **Export to final draft** – A button to compile the user's chosen narrative path into a single formatted document (Markdown, PDF, or plain text) ready for editing or sharing. The export should read smoothly as a complete short story, not a series of choice prompts.

4. Stretch Features (Good-to-Have)

- **Character and world consistency checker** – Before generating a new scene, optionally call the LLM to check character names, locations, and facts from the story so far, and prompt it to maintain consistency. Flag potential plot holes.
- **Multiple genre/tone templates** – Pre-built prompts for different genres (fantasy, sci-fi, romance, horror, thriller) so users get genre-appropriate suggestions without manually specifying.
- **Collaborative branches** – Let multiple users explore the same story and see each other's branches in real time (requires real-time sync, ambitious for 48 hours).
- **AI-assisted editing** – After export, let users send scenes back to the AI for refinement ("Make this dialogue snappier," "Add more description," "Tighten the pacing").
- **Like/dislike feedback** – Let users rate scenes or plot branches, and the AI learns preferences for future generations (requires fine-tuning, probably too ambitious).

5. Suggested Tech Stack

- **Frontend:** React.js or Vue.js (good for interactive UI, branching logic, tree visualization). Svelte is also lightweight.
- **Backend:** Node.js + Express or Python + Flask (fast prototyping, clean LLM integration).
- **Database:** Firebase Firestore (easy auth + document storage for story projects, branches, and scenes) or SQLite/MongoDB for simpler local storage. For MVP, localStorage in the browser works if users don't need to save across devices.
- **LLM API:** OpenAI (GPT-3.5-turbo, ~\$5-10 of free credits per hackathon session, excellent at creative writing) or free: Hugging Face transformers (e.g., mistralai/Mistral-7B or open-source models, slower but zero cost). For narrative quality, OpenAI is recommended.
- **Tree visualization:** react-tree-graph or vis-network (JavaScript library for graph visualization) or D3.js if you want custom styling. Simpler alternative: render as an indented text outline (less flashy but faster to implement).
- **Document export:** pdfkit, jsPDF, or simple Markdown generation + print-to-PDF from the browser.
- **Deployment:** Vercel (frontend), Heroku or Railway (backend), or Firebase for full-stack.

Why these: All beginner-friendly, free or low-cost, and well-documented. React + OpenAI is the fastest path to a polished demo. Tree visualization can be complex, so starting with a simpler UI (indented list or text tree) is wise for a 48-hour build.

6. Suggested Approach / Step-by-Step Plan

1. **Project setup and environment** (30 mins) – Initialize frontend and backend repos. Sign up for OpenAI API (free trial credits) or set up Hugging Face locally. Create folder structure and a basic README. Agree on data structure for stories, branches, and scenes (e.g., a Story object with nested Scenes and Branches).
2. **Design the story data structure** (1 hour) – Define how to represent a story with branches in code. Example structure:

```

Story {
  id, title, premise, genre, tone,
  scenes: [Scene1, Scene2, ...],
  branches: [Branch1, Branch2, ...], // each branch tracks
  which scene it follows and the choice text
  current_path: [scene_id1, scene_id2, ...] // the user's
  current narrative thread
}
Scene {
  id, text (300-500 words), scene_number, parent_branch_id,
  branch_choices: [text1, text2, text3]
}

```

Sketch this out before coding so the backend and frontend agree on the shape.

3. **Build the story premise input form and UI skeleton** (1–1.5 hours) – Create a simple frontend form with fields: Story Title, Genre (dropdown), Tone, Opening Premise (textarea). Add a "Start Writing" button. Below, create a placeholder for the story view (where scenes and choices will appear). Keep styling clean and readable.
4. **Design and test the LLM prompt for opening scene generation** (1.5–2 hours) – Write a detailed prompt that instructs the LLM to generate a compelling opening scene. Example: *"Write the opening scene for a [genre] story with this premise: [premise]. Tone: [tone]. Generate a vivid 400-word scene that establishes setting, character, and hook. Make it engaging and end mid-action or mid-thought to create intrigue."* Test this prompt 5+ times manually with different premises to ensure quality. Refine until you get scenes that feel like a real story opening.
5. **Build the backend scene generation and branch-point endpoints** (2–2.5 hours) – Create two endpoints:
 - **POST /api/stories** – Takes premise/genre/tone, generates opening scene, returns Story object with first Scene.
 - **POST /api/scenes/:storyId/continue** – Takes the current story state and the user's chosen plot direction (which choice they picked). Generates the next scene using LLM, maintaining narrative coherence by passing the full story context. Returns updated Story with new Scene and new branch choices.

For coherence, each LLM call should include: opening premise + full narrative so far + chosen direction. Example prompt: *"Story so far: [full text of previous scenes]. The user chose this direction: [choice summary]. Continue the story with a vivid 400-word scene that follows logically from what came before. Maintain character voices and plot consistency. End with 2-3 new plot direction options."*

6. **Build the scene display and choice UI** (1.5–2 hours) – On the frontend, display the current scene as readable text. Below, show 2–3 clickable choice buttons. Each button triggers the "continue" endpoint. Add visual feedback (loading spinner, scene counter, breadcrumb of choices made so far). Keep it minimal but engaging.

7. **Implement story tree visualization** (2–3 hours) – This is the trickiest part. Start simple: render the story as an **indented text outline** (e.g., "Scene 1 → Choice A → Scene 2 → Choice B → Scene 3"). This is much faster than a full graph visualization. You can click on any scene in the outline to "jump" to that branch and explore alternatives. If time permits, upgrade to a visual tree using `react-tree-graph` or similar, but the text outline is sufficient for MVP and judges will appreciate simplicity.
8. **Implement branch navigation and context preservation** (1.5 hours) – Let users click on scenes in the tree to jump between branches. When they jump, the frontend loads that branch's full context and displays it. Add a "Current Path" indicator showing which route they're following. This requires keeping the full Story state in frontend or backend state, with all scenes and branches accessible.
9. **Build the export to final draft feature** (1.5–2 hours) – Create a button to download the current chosen path as a clean, formatted document. Stitch together all scenes in the current path, remove choice prompts, add smooth transitions between scenes, and format as Markdown or PDF. Include a title page (story title, user name, date). Test the export to ensure it reads as a coherent short story, not disjointed sections.
10. **Test end-to-end, create sample stories, and deploy** (1.5–2 hours) – Test the full flow: enter premise → generate opening → make choices → explore branches → view tree → export. Create 2–3 sample stories (e.g., a mystery, a sci-fi tale, a romance snippet) to demo. Deploy frontend and backend. Prepare a clean README and demo walkthrough. Have backup sample stories hard-coded in case LLM is slow during demo.

7. Sample Data / APIs to Use

- **LLM APIs:**
 - **OpenAI:** Use `gpt-3.5-turbo` via REST API or Python library. Free trial (~\$5-10) provides 20–30 full story branches (each branch = 2–3 LLM calls for scenes + choice generation).
 - **Hugging Face:** Use `transformers` library locally. Models like `mistralai/Mistral-7B` or `meta-llama/Llama-2-7b-hf` work for creative writing. No cost, but slower (~30-60 secs per generation on CPU).
- **Sample story premises (for testing):**
 1. Genre: Mystery. Premise: "A detective finds a coded message in an old library book. The sender is someone she thought was dead."
 2. Genre: Science Fiction. Premise: "An astronaut wakes from cryosleep to find the ship's AI has gone rogue and is heading toward a black hole."
 3. Genre: Fantasy. Premise: "A thief discovers a magical amulet in a pawn shop. It whispers secrets that are destroying her peace of mind."
 4. Genre: Horror. Premise: "A woman moves into a new apartment and realizes her neighbor has been watching her through the walls."
- **Mock story structure (for testing UI before LLM is ready):**

```

{
  "story_id": "story-001",
  "title": "The Hidden Door",
  "premise": "A detective finds a coded message in an old library book.",
  "genre": "Mystery",
  "tone": "Noir",
  "scenes": [
    {
      "id": "scene-1",
      "text": "Detective Sarah Chen had been staring at the rare books collection for three hours when her fingers brushed something odd—a piece of paper folded inside a 1920s copy of 'The Big Sleep.' The handwriting was shaky but deliberate: 'She's not dead. Check the old asylum on Riverside. -M'...",
      "scene_number": 1,
      "branch_choices": [
        "Call backup and head to the asylum immediately",
        "Research the asylum's history first; this might be a trap",
        "Try to trace where this book came from"
      ]
    }
  ],
  "branches": [],
  "current_path": ["scene-1"]
}

```

8. Key Challenges to Anticipate

1. **LLM narrative coherence across branches** – As users explore multiple paths, the AI might lose track of established characters, locations, or plot points, or generate scenes that contradict earlier events. **Tip:** Always pass the *full narrative history* (all previous scenes in the current path) to each LLM call. Include a system prompt reminding the AI to maintain consistency: *"Keep character names and details consistent with the story so far. Avoid contradicting earlier events."* Test a few multi-branch stories manually to catch incoherence early. If the AI frequently contradicts itself, add a validation step where you call the LLM to check consistency before returning a new scene.
2. **Tree visualization complexity** – A full graph visualization (using D3.js or vis-network) can be complex and time-consuming. **Tip:** Start with a **simple indented text outline** (e.g., "Scene 1 > Choice A > Scene 2 > Choice B > Scene 3") that users can click to jump between branches. This is much faster to build, still shows the narrative branching, and judges will appreciate the simplicity. Upgrade to a visual tree *only* if you have 4+ hours remaining.
3. **LLM API cost and latency** – Each scene generation costs a few cents and takes 5–10 seconds. Multiple branches = multiple calls. **Tip:** Use GPT-3.5-turbo (cheaper than GPT-

- 4). Cache or batch requests if possible. Show clear loading spinners so users know the AI is thinking. Have a few pre-generated sample stories hard-coded as a fallback for demo if the API is slow.
4. **Export stitching and readability** – Exporting a branching story as a single linear narrative requires thoughtful transitions and editing to avoid jarring jumps. **Tip:** After stitching scenes together, optionally use the LLM to generate smooth transitional sentences between scenes (e.g., "The next morning, Sarah made her decision..."). Alternatively, let users manually add transitions in the exported document. Test the export with a few branches to ensure it reads smoothly, not choppy.

9. What a Good Demo Looks Like

The demo should show a user entering a story premise (e.g., "A detective finds a mysterious message in an old book. Mystery genre. Noir tone."). Within seconds, a compelling 400-word opening scene appears, establishing the detective, the setting, and tension. Below, three plot choices appear ("Investigate the clue immediately," "Research the book's history," "Tell a partner first"). The user clicks one, and the AI generates the next scene (300–500 words) that logically follows their choice, introducing new details and ending with new choices. The user explores 2–3 branches by clicking choices and jumping back to explore alternatives. The evaluator should see an indented text tree showing all explored branches (e.g., "Scene 1 → Choice A → Scene 2 → Choice B → Scene 3"). Finally, the user clicks "Export," and a clean PDF downloads showing their chosen narrative path stitched together as a polished 1500–2000 word short story—no choice prompts, smooth transitions, ready to read. Judges should think: "I could actually use this to brainstorm a story—and the export is a real draft I can refine."